Python Engineering - 01_python_bootcamp

General content index and review map

This document is the navigation map for `01_python_bootcamp`. It does not replace class notes: it helps locate **where a concept is introduced** and which week to revisit.

```
01_python_bootcamp/
|-- week_01_logic/
|-- week_02_data_structures/
|-- week_03_modules_and_coordinates/
`-- week_04_oop_files_testing/
```

week_01_logic

Logic fundamentals and equation -> algorithm translation

Week 01 builds the mental foundation of the course: take an engineering problem, identify inputs, outputs and equations, and translate them into executable steps. We begin with variables and expressions, then add decisions, loops and reusable functions.

Objectives

- Translate engineering equations into step-by-step algorithms.
- Work with numerical variables, expressions and results.
- Control decisions with if / elif / else.
- Automate repeated calculations with loops.
- Split programs into reusable and testable functions.

week_02_data_structures

Data structures and numerical processing

Week 02 focuses on how data is organized. We move from isolated variables to lists, tuples, dictionaries, sets and NumPy arrays. The goal is to store trajectories and configurations, select data, apply vectorized operations and think about datasets in terms of rows, columns and axes.

Objectives

- Choose between list, tuple, dict, set and np.ndarray.
- Understand mutability, indexing, slicing and membership.
- Use shape, ndim, size, dtype and axis in NumPy.
- Apply broadcasting, reshape, transpose and array stacking.
- Filter and analyze data with masks and NumPy functions.

week_03_modules_and_coordinates

Modular architecture and engineering tools

Week 03 turns isolated scripts into organized programs. We separate responsibilities across modules, import functions, control execution with main and reuse code. We then extend function syntax and build vector-algebra and coordinate-transformation tools.

Objectives

- Split code into modules with clear responsibilities.
- Understand imports, __name__ and the main() pattern.
- Use positional, keyword, default, *args and **kwargs arguments.

- Apply rotation matrices and vector algebra with NumPy.
- Build a reusable engineering toolkit.

week_04_oop_files_testing

Objects, persistence, errors and software quality

Week 04 introduces tools that make programs look more like real software. We model entities with classes and objects, persist data with File I/O, handle exceptions, debug code and close the bootcamp with testing and project structure.

Objectives

- Understand class, object, attributes, methods, __init__ and self.
- Save and recover results using files and CSV.
- Separate physical model, persistence and orchestration.
- Handle errors with exceptions and debug with tracebacks/tools.
- Prepare verifiable code and a maintainable project structure.

Quick map: where should I review a concept?

If I need to review...	Go first to...
variables, numeric types, operators	Week 01 - Class 01
if / elif / else, comparisons	Week 01 - Class 02
loops, range, accumulators, Euler	Week 01 - Class 03
def, parameters, return, scope	Week 01 - Class 04
lists / tuples	Week 02 - Class 05
dict / set / configuration	Week 02 - Class 06
NumPy arrays, shape, ndim, axis, reshape, vstack	Week 02 - Class 07
masks, filtering, np.where, np.diff, np.allclose	Week 02 - Class 08
modules, imports, main, __name__	Week 03 - Class 09
*args, **kwargs, keyword arguments/unpacking	Week 03 - Class 10
rotation matrices and @	Week 03 - Class 11
norm, unit vector, dot product, projection	Week 03 - Class 12
class, object, self, attributes, methods	Week 04 - Class 13
open, Path, CSV, read/write	Week 04 - Class 14
try/except, raise, traceback, debugger	Week 04 - Class 15
testing and project structure	Week 04 - Class 16

Recommended use

When you forget a function or concept, do not force yourself to remember every syntax detail. Use this index to locate the class, open its PDFs and return to the corresponding exercise. The bootcamp goal is not memorizing Python: it is knowing **what tool exists, why it is useful and where to recover it**.